BIT502 Fundamentals of Programming Assessment 2

Weighting

30%

Learning outcomes

1 Create simple applications using fundamental programming logic, constructs and mathematical operations to solve business problems.
3 Utilise a range of software tools commonly used during the software development life cycle to create, test and document simple working code.

Instructions

Complete and submit your assessment according to the Open Polytechnic's Assessments webpage. This includes information on academic integrity, formatting, word limits and referencing.

- Include your name, student number and the assessment number.

Submission

- Submit your assessment in two files.
- Submit your work through your iQualify course.
- Emailed assessments will not be accepted.
- You will receive an automated notice advising you of your successful submission.

By submitting your assessment, you confirm that it is your own, original work.

© The Open Polytechnic of New Zealand

Introduction

In this assessment, you will build a GUI application using Python and the GUI library, Tkinter. You should refer to the example GUI design and base your application on it: Appendix B Example GUI Design. Your GUI application must address the needs of the business in the case study. You may also need to consult the information available in Appendix A Membership Form and Appendix C Sample Calculations.

Note: We recommend using Visual Studio Code instead of Jupyter Notebook as the code editor to complete this assessment.

Note: While it is highly advisable to utilize this template or build upon it, you are welcome to create your own GUI form, provided it aligns with the tasks and requirements specified in this assessment. However, please refrain from using any additional libraries or dependencies beyond Tkinter.

Before you submit

- Your documentation can be submitted as a PDF document.
    - Include your name, student number, assessment number and page numbers on your Word document.
- The code file should be submitted as a zip file.
    - Include your course code (BIT502), name, student number, and assessment number as a comment, at the top of your code.
- Ensure your zip file includes:
    - your Python file.

  After zipping your file, reopen it to make sure it works and contains the correct file.

Read the following case study and complete the tasks. Refer to the Marking Schedule for specific information about marking.

© The Open Polytechnic of New Zealand

Case study

A local bookstore, The Aurora Archive, has been in business for several months and is seeing a steady increase in members. In addition to selling books, they have a few services that require the use of a membership subscription; these include:

- access to a private reading area
- encouraging kids to adopt reading as a lifelong hobby
- various book and reading related activities.

The owners have asked you to develop some programs to help improve their administration service to save them time and reduce errors.

Automating membership processes

As the store's membership grows, keeping track of client details is becoming more difficult. They have tested a basic computer program to improve the calculation of membership plans and costs but still mainly rely on an old-fashioned method using pen and paper. This includes:

- giving a paper form to the customer
- staff members calculating fees manually
- customers receiving a copy of the membership form, with another copy filed away.

Currently, the paper-based registration method is quite time-consuming for the staff. This means that the staff members must spend extra time making sure the information is correct and working with the clients to fix any mistakes. Since this process is done on a paper form, they often must restart with a new form if there are mistakes or errors.

The previous program you created was found to be quite helpful and convenient for the bookstore, so the owners are considering moving towards a fully digital sign-up process. The proposed application would allow clients to input their details from the existing form and calculate membership costs.

The store requires a program that would:

- assist clients in the registration process by providing all the necessary options
- show clients the total costs before they register to make sure they have chosen the right plan for them
- verify that clients have entered data correctly
- store the client information digitally for access later.

Moving to a digital format would make the membership process more efficient, save staff time, and provide accurate information to clients.

© The Open Polytechnic of New Zealand

Task 1: Developing the membership application

As the programmer, your task is to apply and complete the functionality of an application. You will build the GUI application for The Aurora Archive using the designed GUI – Appendix B Example GUI Design.

The application is a membership form that will calculate the membership costs and write the new membership details in a text file.

Part A: Finishing the user interface

For this program, the GUI has already been created for you; you have been tasked with writing the code to provide functionality. The GUI is provided in Appendix B.

Use the UI widgets functionality to build the application. Ensure the GUI is working correctly and set your project up as you like, adding in comments as necessary.

This application only requires a single form, the entire application can be made within one Python code file.

Make modifications to the UI to improve the functionality where you feel necessary, but the overall ability of the form should not change. For example, you may change the colours, fonts, or resize widgets.

The GUI should be fully functional: the text boxes, radio buttons, checkboxes, and buttons should work as expected and provide a good user experience.

Note: While it is highly advisable to utilize this template or build upon it, you are welcome to create your own GUI form, provided it aligns with the tasks and requirements specified in this assessment. However, please refrain from using any additional libraries or dependencies beyond Tkinter.

(10 marks)

Part B: Pricing and calculations

Calculation should be performed based on the options selected; if a required option has not been selected yet, the application should warn the user and cancel calculation.

Calculation is performed when the user clicks the calculate button or automatically as the options are selected.

If all required options have been selected, the application should check the selected options on the interface and calculate:

- membership cost

© The Open Polytechnic of New Zealand

- total of the optional extras
- annual and library card discount
- payment type discount.

Once the application has calculated each option, it will be able to use those values to calculate:

- total monthly cost
- total monthly optional extra costs
- total monthly discount
- an approximate weekly cost to help the client decide if it is right for their budget.

Notes:

- The final cost is the base membership cost, including extras, deducting discounts.
- Calculations should be based on the options selected by the user. Using separate variables to track these individually will be beneficial.
- The year is considered to be 52 weeks exactly. A month is considered to be four weeks exactly.
- The discount for the library card is applied first, before the annual discount.

Pricing

The pricing section of your application should include the following. Prices shown are all monthly costs.

- The three membership types and costs are:
    1. Standard: $10 per month.
    2. Premium: $15 per month.
    3. Kids: $5 per month.
- After the client chooses the membership type, select the payment plan:
    - Monthly, no discount is provided
    - Annual/Yearly, if this is selected, they receive 1 month free for the membership cost, but this does not include extras.
- If the client indicates they are part of the local library and have a library card, they receive a 10% discount on the base membership cost (the standard, premium, or kids option) and any extras selected. They must enter a valid ID card; the ID will always be a five-digit number.
- The client can then select from any of the following extras.
    - Book rental is $5 per month.
    - Private area access is $15 per month.
    - Monthly booklet is $2 per month.

© The Open Polytechnic of New Zealand

- Online ebook rental is $5 per month.

Note: The discount for selecting an annual payment plan is applied after the library card discount. For example, the standard plan with a library card will cost $9 due to the 10% discount; if the annual plan is selected, they will receive a $9 discount for the 1 month free.

Calculated totals

Use Appendix C Sample Calculations to help with calculating different membership scenarios.

When calculating, the application should display the sub-totals and totals to the user.

1. Membership plan cost: The monthly cost based on the membership type they selected (from the standard, premium, kids' options) without discounts.
2. Total extras: The total monthly cost of all the extras the client has selected. If they have not selected any extras, the total will be 0 without discounts.
3. Total discount: The total monthly discount provided. Includes library discount and annual discount. Display as a monthly total.
4. Weekly cost: Show how much the membership will cost if broken down on a weekly basis. This is for the convenience of the user; it should not be used in calculations.
5. Payment due: The amount required to be paid, based on their payment plan and calculated costs. This will be the total cost minus any discounts. Indicate if this is a yearly or monthly payment, for example: “"Payment Due: $342 yearly"

(30 marks)

Part C: Updating the interface

After the calculations are made, update the interface of the application to display the totals. The values should be clear and rounded to two decimal places since it is showing currency. For example, 10.28412 should be shown as 10.28.

(5 marks)

Part D: Validation and error checking

Before submitting the form and writing to the file, the application should validate any values the user has entered and perform error checking to ensure the values are correct. The application should provide an appropriate warning message to the user and not write to the file if there are errors found.

The following criteria must be considered:

- No text field should be empty.
- No radio button component should go unselected.
- The first and last name fields should not contain any number.
- You do not need to validate email addresses.

© The Open Polytechnic of New Zealand

The best place to perform this validation and error checking is when the user attempts to submit the form.

Additionally, you can indicate which fields are required to complete by changing the interface, such as placing a red asterisk next to incomplete fields.

(10 marks)

Part E: Writing the client_data text file

Once validation and error checking have been performed, the application should write the user's selections into a text file.

For example:

First name: Mary

Last name: Wilson

Address: 54 Smith Road, Waterloo, Wellington

Mobile: 8376222

Membership type: Standard

Extras: Monthly booklet

Payment plan: Monthly

Library card: No

Library card ID: ---

Base membership cost: $10

Extra cost: $2.00

Total discount: $0

Total cost: $12

Weekly cost: $2.31

Payment due: $12

The file name for this text file should be "members_data.txt".
Write this file to the same directory the program is being run from.

Note: The following points need to be considered when coding this part.

- Performing the data validation and error checking first.
- Opening a connection to the text file.
- Saving the data into the text file.
- Closing a connection to a text file.
- Not overriding the previously saved data inside the text file (append to the file).

Note: Regarding the text file, please follow these instructions:

© The Open Polytechnic of New Zealand

- Place the text file in the same folder as your Python source code file.
- When submitting your assessment, ensure that you include the text file along with your code file.
- When defining the text file path in your code, do not use an absolute path (e.g., C:\temp). Instead, use a relative path.

(10 marks)

Part F: Form completion

Once the user has submitted the form and the data has been successfully written into the file, the following actions need to take place.

1. Let the user know the data has been saved successfully by displaying a message box or similar prompt.
2. The form is cleared to allow for another user to enter their data.

Additionally, update the interface to include a new button “Reset” that will allow the user to reset the form manually if a user leaves without completing the form; it should confirm with a prompt before resetting the form.

(10 marks)

Part G: Automatic calculation

With the basic functionality of the program complete, the client has requested an update if it is possible to include it during this development cycle.

While the calculate button is useful, ideally the form could update itself as the user enters information rather than needing to click the button.

Update the program to automatically calculate the totals whenever the user changes one of the options related to the cost. For example, when a new extra is selected or duration is changed, the totals will automatically be calculated and displayed without requiring the user to click the calculate button.

(5 marks)

© The Open Polytechnic of New Zealand

Quality control

Revise your code once you are finished to ensure good programming practices have been followed, such as meaningful variable names and organised code.

Essential functions of your application

Using the GUI file provided, you will complete the functionality of the application. The user of the program (the client) must be able to:

- open the GUI and enter their personal details
- select the membership options they would like
- the totals are calculated either automatically if automatic calculation was completed or the user must click the “calculate” button to view the totals of the options they have selected
- consider whether the options and correct, changing the options as they like and repeating the calculation
- submit the form, which will validate their personal information and provide a warning or continue and write their information and selections to a text file with their correct information based on the options selected.

The application should be simple to use. Users should be able to quickly calculate their membership and decide if they want to sign up. Improve the user experience by picking suitable colours and positioning elements appropriately in the GUI.

Good programming practices

Your code should follow good programming practices, including:

- organising the code in a clear manner
- using meaningful variable naming
- minimising duplicate code
- using your own methods where sensible
- performing validation where necessary
- handling unexpected input or user mistakes appropriately
- ensuring the program does not crash or produce errors
- avoiding hard-coding values where possible
- adding comments to explain the complex sections of the code.

Although well written code should not require much if any commenting, as the programs may undergo future development you have been asked to provide some comments to help explain any code that may not be obvious, such as calculations and formula in the code. You should not comment on every line.

Ensure your program is free from errors and compiles correctly.

(Task 1 total: 80 marks)

© The Open Polytechnic of New Zealand

Task 2: Testing and documentation

Consider the software development lifecycle and the importance of the testing phase. Once the development of your application is complete, you must carry out an extensive evaluation of the completed software.

Part A: Test plan

Once you have finished building the application, complete the following tasks:

1. Create a test plan featuring a minimum of eight and a maximum of fifteen unique test items. The plan should be presented as a table in a Word document. Give your table an informative title.
2. The test plan should expand on the user acceptance testing and cover all requirements and acceptance criteria given in the case study.
3. Several of your tests should involve functional testing to ensure that the calculations are giving accurate results.

Set up your table as follows.

Table 1. Test plan

Test Item	Expected results	Actual output	Actions required

1. Carry out a series of extensive user tests according to the test items in your table.
2. Complete the columns for 'Actual output' and 'Actions required' for each test item.
3. Failed tests can be tested again after fixing the issues found.

(15 marks)

Part B: Applying the steps in the software development lifecycle

Describe the steps you took in the software development process to ensure the client gets a viable product. Your descriptions should be easy to understand by the client and must cover these steps.

- Requirement analysis: Describe what you did during this stage.
- Design: Outline the key decisions you made.
- Implementation: Explain how you put the plan into action, highlighting key decisions.
- Testing: Discuss the tests you carried out and what you learned from them. Highlight the key decisions.

Remember to do the following.

- Keep descriptions brief and clear.

© The Open Polytechnic of New Zealand

- Explain how you have made the software easy to use. You might also want to mention user testing results.
- Comment on benefits to the client.

Keep the document under a page so the client can quickly understand the process. You do not need to talk about the evolution stage.

Software and tools

Through the development of your program, you will have used a range of software and tools. In your document, create a new section to list the software used along with a brief description of how it was used. Include any software that has assisted in the development and testing, for example: IDE and plugins, text editors, spreadsheets, calculators, image editors, and software used to plan or prototype.

For each piece of software or tool you have used, provide:

- the name of the software or tool
- a short 1-2 sentence description of the purpose of the software or tool and how you utilised it.

Format your document appropriately and include your name and student ID.

(Word count guideline: 400 words)

(5 marks)

(Task 2 total: 20 marks)

(Total: 100 marks)

© The Open Polytechnic of New Zealand

Marking Schedule

Task 1: Part A	8 – 10 marks	6.5 – 7.5 marks	5 – 6 marks	4 – 4.5 marks	1 – 3.5 marks
Finishing the user interface	User interface produced using GUI library. All items included, with no errors present. Radio buttons have been given default values. Improvements to the GUI have been made to customize and make it more user friendly. Visual warnings for incomplete fields have been used and are visible only when necessary. Good user-interface practice is applied.	User interface produced using GUI library. All items included, with no errors present. Radio buttons have been given default values. Good user-interface practice is applied. Minor customization has been given to improve the experience.	User interface produced using GUI library. Interface is fully usable. Some minor issues may exist but do not interfere with functionality. Good user-interface practice is applied with minor inaccuracies.	User interface produced using GUI library. Parts of the GUI are not yet working.	User interface produced using GUI library. Not all items included with significant errors present.
Task 1: Part B	**24 – 30 marks**	**19.5 – 23.5 marks**	**15 – 19 marks**	**12 – 14.5 marks**	**1 – 11.5 marks**
Pricing and calculations	All required calculations are present, functional and accurate. Good programming practices have been followed throughout; variables are meaningfully named, values are not hard-coded, methods are made where sensible.	All required calculations are present and functional. The calculations are correct without errors. All totals are calculated. Some errors exist or some good practices have not been followed.	All required calculations are present and functional. Some errors in coding and functionality are present but the basic functions are performed.	Coding has been included inside the calculate button. All required calculations are present. Significant errors in coding and functionality are present.	Coding has been included inside the calculate button. Not all required calculations are present. Significant errors in coding and functionality are present.

© The Open Polytechnic of New Zealand

Task 1: Part C	4 – 5 marks	3.5 marks	2.5 – 3 marks	2 marks	1 – 1.5 marks
Updating the interface	All calculations are accurately displayed on the interface. Values have been rounded correctly. Good programming practices have been followed throughout.	All calculations are displayed on the interface. Some errors exist or some good practices have not been followed.	All calculations are displayed on the interface. Some errors are present, but the values are all updated on the interface.	All calculations are displayed on the interface. Significant errors in the display exist. Some of the calculations are displayed on the interface but some are missing, or the calculations are displayed using a prompt or another method.	Displaying the calculations has been attempted but not functional yet.
Task 1: Part D	**8 – 10 marks**	**6.5 – 7.5 marks**	**5 – 6 marks**	**4 – 4.5 marks**	**1 – 3.5 marks**
Validation and error checking	All fields are validated and there is thorough error checking performed. A visual indicator of incomplete fields has been added without the use of a prompt.	All fields are validated and there is thorough error checking performed. Warnings are provided by a prompt or similar method.	Most user input is validated and there is various error checking performed.	Some error checking or validation is attempted with partial functionality.	Some error checking or validation has been attempted but cover little or are not functioning.
Task 1: Part E	**8 – 10 marks**	**6.5 – 7.5 marks**	**5 – 6 marks**	**4 – 4.5 marks**	**1 – 3.5 marks**
Writing the client_data text file	Submit functionality is working well. Data is written in a well-organized format; no data is missing. Good programming practices are followed throughout, and appropriate error checking is performed.	Submit functionality is working well. Data is written in a well-organized format; no data is missing. Data is appended. Some good programming practices have been followed.	Submit functionality is working, allowing the user to write to the file with the basic information. Minor errors, information may be missing yet, or the file is not appended to.	Submit partially works with some functionality but mostly incomplete.	Submit button has been coded. Significant errors in coding and functionality exist. No data is saved. Submit functionality has been started but not yet functional.

© The Open Polytechnic of New Zealand

Task 1: Part F	8 – 10 marks	6.5 – 7.5 marks	5 – 6 marks	4 – 4.5 marks	1 – 3.5 marks
Form completion	All sections of the form are cleared of all data upon completion of user submission, preventing submitting the same information twice. Radio buttons are reset to their original value. Visual indicators for fields that were missing upon submission are cleared.	All sections of the form are being cleared upon submission. Most values are reset to their default option. Minor errors may be present.	Most of the form is being cleared upon submission. There may be some minor issues or not all values are reset to their default.	At least one section of the form is being cleared upon submission, indicating the functionality exists to expand upon.	Some form clearing has been attempted in at least one section of the form, but some issues exist.
Automatic Calculation	**4 – 5 marks**	**3.5 marks**	**2.5 – 3 marks**	**2 marks**	**1 – 1.5 marks**
	Any changes to options update all of the total values correctly.	Almost all options are correctly updating the total values.	Interface has some elements update automatically. Some totals are not updated or are missing functionality.	Some updates occur when changing interface elements but much of the functionality is missing.	Some attempts have been made but the functionality is not working.
Task 2: Part A	**12 – 15 marks**	**10 – 11.5 marks**	**7.5 – 9.5 marks**	**6 – 7 marks**	**1 – 5.5 marks**
Test plan	Evidence of user acceptance testing provided. No errors in structure present. Test plan accurately covers all requirements. At least 15 tests have been performed with a least 7 involving calculation accuracy. Failed tests have	Evidence of user acceptance testing provided. No errors in structure present. Test plan covers all requirements. Some ambiguity in how some requirements are covered exists. At least 12 tests have been performed with	Evidence of user acceptance testing provided. Test plan covers all requirements. Some ambiguity in how some requirements are covered exists. At least 8 tests have been performed.	Evidence of user acceptance testing provided. Significant errors in structure present. Test plan covers all requirements. Some ambiguity in how all requirements are covered exists.	Evidence of user acceptance testing provided. Significant errors in structure present. Test plan does not cover all requirements.

© The Open Polytechnic of New Zealand

	been repeated until successful.	at least 5 involving calculation accuracy.			
Task 2: Part B	**4 – 5 marks**	**3.5 marks**	**2.5 – 3 marks**	**2 marks**	**1 – 1.5 marks**
Applying the steps in the software development life cycle	Description of steps to ensure the production of a viable product. Phases of software development accurately provided. Explanation as to how the application was designed to be easy to use is provided. Explanation is justified with reference and comparison to accessibility principles. A complete list of software used during development is included along with clear, appropriate descriptions.	Description of steps to ensure the production of a viable product. Phases of software development accurately provided. Explanation as to how the application was designed to be easy to use is provided. A complete list of software used during development is included along with appropriate descriptions.	Description of steps to ensure the production of a viable product. Phases of software development provided with minor inaccuracies. Explanation as to how the application was designed to be easy to use is provided, but ambiguity around main points exists. A complete list of software used during development is included along with adequate descriptions.	Description of steps to ensure the production of a viable product. Phases of software development provided, but with significant inaccuracies. Explanation as to how the application was designed to be easy to use is provided, but ambiguity around main points exists. Software used during development is listed, but descriptions have not been included or are difficult to understand.	Description of steps to ensure the production of a viable product. Phases of software development provided. but with significant inaccuracies. Explanation as to how the application was designed to be easy to use is not provided. Software used during development is not listed or is inappropriate, and descriptions have not been included.

© The Open Polytechnic of New Zealand